Lecture 5. Evaluation measures, Imbalanced classes, Cost-Sensitive Classification

Data mining, prof. Biljana Tojtovska Ribarski
FCSE, Skopje

Contents

1 Evaluation measures, Imbalanced classes, Cost-Sensitive Classification **1**
1.1 Introduction: What Makes a Classifier Good? . . . 1
1.2 The Class Imbalance Problem . . . 2
1.3 Confusion Matrix and Basic Outcomes . . . 2
1.4 Measures of Classification Performance . . . 3
1.5 Why Accuracy Alone Can Be Misleading? . . . 4
1.6 Comparisons of Different Measures . . . 5
1.7 Thresholds and Score-Based Classification . . . 5
1.8 ROC Curves . . . 7
1.9 How To Construct a ROC Curve? . . . 9
1.10 Using ROC and AUC for Model Comparison . . . 10
1.11 ROC Curves versus Precision–Recall Curves . . . 12
1.12 Handling the Imbalanced-Class Problem . . . 12
1.13 Cost-Sensitive Classification . . . 13
1.14 Practical examples: when the best statistical model is not the best business choice 15
1.15 Sampling-Based Approaches . . . 15
1.16 Which Measure Should We Use? . . . 16
1.17 What To Do in Practice . . . 17
1.18 Summary . . . 18
1.19 Exercises . . . 18

1 Evaluation measures, Imbalanced classes, Cost-Sensitive Classification

The evaluation ideas in this chapter apply to *all* classification methods, not only to decision trees. They are relevant for logistic regression, naive Bayes, k-nearest neighbors, support vector machines, trees, random forests, boosting methods, neural networks, and other classifiers.

1.1 Introduction: What Makes a Classifier Good?

So far, classification has mainly been discussed as a predictive task: given an input x, we want to predict its class label correctly. But once classification models are used in practice, an important question appears: what does it mean for a classifier to be *good*?

The answer is not always "the classifier with the highest accuracy." In many real problems:

- one class is much rarer than the other;
- false positives and false negatives do not have the same practical consequences;

- we care not only about the final class label, but also about scores, ranking, thresholds, or cost.

This is why evaluation is not a small technical detail. It is part of the classification problem itself.

Insight: Central message

A classifier is not judged well by one universal number. What counts as a good classifier depends on class imbalance, the relative importance of different errors, and the practical goal of the task.

1.2 The Class Imbalance Problem

Definition: Class imbalance

A classification problem has *class imbalance* when one class occurs much more frequently than another.

Many important classification problems are imbalanced. Examples include fraud detection, intrusion detection, defective-product detection, rare-disease screening, and rare-event monitoring. In such settings, the rare class is often the class that matters most.

Example: Why imbalance matters?

Suppose we are building a fraud detector for a batch of 1000 transactions. Assume that 990 are normal and only 10 are fraudulent. If a classifier predicts every transaction as `No Fraud`, then its accuracy is

$$\frac{990}{1000} = 0.99.$$

This looks excellent, but the classifier fails to detect a single fraudulent transaction. In practice, such a classifier is useless.

High accuracy does not automatically mean good classification performance. On imbalanced data, accuracy can be dominated by the majority class and completely hide failure on the rare class.

1.3 Confusion Matrix and Basic Outcomes

The confusion matrix is the basic summary of classification outcomes in the binary case.

	Predicted Yes	**Predicted No**
Actual Yes	TP	FN
Actual No	FP	TN

Definition: Confusion matrix entries

In binary classification:

- **TP** (true positive): actual positive, predicted positive;
- **FN** (false negative): actual positive, predicted negative;
- **FP** (false positive): actual negative, predicted positive;
- **TN** (true negative): actual negative, predicted negative.

Each evaluation measure emphasizes a different part of this table. Therefore, before interpreting any metric, one should always know which entries of the confusion matrix it depends on.

1.4 Measures of Classification Performance

Definition: Accuracy and error rate

The **accuracy** of a classifier is the proportion of correctly classified observations:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

The **error rate** is the proportion of misclassified observations:

$$\text{Error rate} = \frac{FP + FN}{TP + TN + FP + FN} = 1 - \text{Accuracy}.$$

Definition: Precision, recall, specificity, and related measures

Precision (positive predictive value) is the proportion of predicted positives that are truly positive:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Recall (also called *sensitivity* or *true positive rate*) is the proportion of actual positives that are correctly detected:

$$\text{Recall} = \text{Sensitivity} = \text{TPR} = \frac{TP}{TP + FN}.$$

Specificity (also called *true negative rate*) is the proportion of actual negatives that are correctly identified:

$$\text{Specificity} = \frac{TN}{TN + FP}.$$

False positive rate is

$$\text{FPR} = \frac{FP}{FP + TN} = 1 - \text{Specificity}.$$

False negative rate is

$$\text{FNR} = \frac{FN}{FN + TP} = 1 - \text{Recall}.$$

The $\mathbf{F_1}$ **score** is the harmonic mean of precision and recall:

$$F_1 = \frac{2\,(\text{Precision})(\text{Recall})}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}.$$

Remark: Interpretation

Instead of repeating the formulas separately every time, it is useful to remember the practical question behind each measure:

- **Accuracy:** how often is the overall prediction correct?
- **Precision:** among the cases predicted positive, how many are truly positive?
- **Recall:** among the truly positive cases, how many did we detect?
- **Specificity:** among the truly negative cases, how many did we leave negative?
- **$\mathbf{F_1}$:** how well do we balance precision and recall in one number?

Remark: Why different measures disagree?

The measures emphasize different parts of the confusion matrix:

- precision depends on TP and FP;
- recall depends on TP and FN;
- specificity depends on TN and FP;
- F_1 depends on TP, FP, and FN, but not directly on TN.

This is why two classifiers can have similar accuracy but very different practical usefulness.

Remark: Connection with statistical testing

In the language of hypothesis testing, the false positive rate is often denoted by α, while the false negative rate is often denoted by β. Then:

$$\alpha = \text{FPR}, \qquad \beta = \text{FNR}, \qquad 1 - \beta = \text{Recall}.$$

So *power* corresponds to sensitivity/recall in the binary-classification setting.

1.5 Why Accuracy Alone Can Be Misleading?

Accuracy is easy to understand and widely used. It is often useful when:

- the classes are not severely imbalanced;
- false positives and false negatives are similarly important.

But when rare positives matter, accuracy alone can be misleading.

Example: High accuracy can hide poor rare-class performance

Consider the confusion matrix

	Predicted Yes	**Predicted No**
Actual Yes	10	0
Actual No	10	980

Then

$$\text{Precision} = \frac{10}{20} = 0.5, \qquad \text{Recall} = \frac{10}{10} = 1, \qquad F_1 \approx 0.67, \qquad \text{Accuracy} = 0.99.$$

Now compare it with

	Predicted Yes	**Predicted No**
Actual Yes	1	9
Actual No	0	990

Then

$$\text{Precision} = 1, \qquad \text{Recall} = 0.1, \qquad F_1 \approx 0.18, \qquad \text{Accuracy} = 0.991.$$

The second classifier has slightly *higher* accuracy, but it is far worse at detecting the positive class.

Warning: A practical lesson

When the positive class is rare and important, reporting only accuracy can lead to the wrong conclusion. The confusion matrix and additional measures must also be reported.

1.6 Comparisons of Different Measures

Example 1. Same recall, very different precision

Consider the confusion matrices

	Predicted Yes	**Predicted No**
Actual Yes	40	10
Actual No	10	40

	Predicted Yes	**Predicted No**
Actual Yes	40	10
Actual No	1000	4000

Both classifiers have recall 0.8, but their precision is very different:

$$0.8 \quad \text{versus} \quad \frac{40}{1040} \approx 0.04.$$

So "same recall" does not imply similar usefulness. A model may detect many positives and still be practically poor if it creates too many false alarms.

Example 2. Same precision, different ROC behavior

Consider the following three confusion matrices summarized only by their key counts:

Case	TP	FN	FP	TN	Precision
A	10	40	10	40	0.5
B	25	25	25	25	0.5
C	40	10	40	10	0.5

For these three cases:

Case A: TPR $= 0.2$, FPR $= 0.2$, Case B: TPR $= 0.5$, FPR $= 0.5$, Case C: TPR $= 0.8$, FPR $= 0.8$.

None of the three cases is universally best, because they all have the same precision and all lie on the ROC diagonal, so the more useful choice depends on the application-specific cost of false negatives versus false positives (e.g., A if we want fewer positive predictions overall, C if missing positives is much worse, and B only as an intermediate compromise)

1.7 Thresholds and Score-Based Classification

Many classifiers do not only output a class label. They may also output:

- a score;
- an estimated probability;
- a confidence value;
- a ranking of cases from most suspicious to least suspicious.

A final class label is then obtained by applying a threshold. In binary classification, one often predicts positive if the score is above some threshold t.

Changing the threshold changes the confusion matrix and therefore changes precision, recall, specificity, and the false positive rate.

Remark: Threshold tradeoff

If the threshold is lowered, the classifier predicts positive more easily. This often increases recall but may also increase false positives. If the threshold is raised, the classifier becomes more conservative, which often improves precision or specificity but may miss more positives.

Example: Different classifiers may provide different outputs

Logistic regression and naive Bayes naturally produce estimated class probabilities. Support vector machines usually produce a score or margin rather than a probability by default. In some applications, such as fraud review or patient triage, the most important output is not a hard label first, but a ranking from highest estimated risk to lowest.

Example: Why scores are useful: a screening example

Consider a first-stage hospital screening system for a serious disease. For each patient, the classifier outputs a risk score between 0 and 1, where larger values mean higher estimated risk.

Patient	P_1	P_2	P_3	P_4	P_5	P_6	P_7	P_8
Score	0.92	0.81	0.74	0.63	0.52	0.47	0.26	0.08
True class	+	+	−	+	−	+	−	−

The goal is not only to assign a final class label, but also to decide *how aggressively* the system should refer patients for further testing.

If the threshold is set to $t = 0.75$, then only P_1 and P_2 are predicted positive. This gives

$$TP = 2, \quad FP = 0, \quad FN = 2, \quad TN = 4.$$

Hence

$$\text{Precision} = 1, \qquad \text{Recall} = \frac{2}{4} = 0.5.$$

This threshold is conservative: it avoids false alarms, but it misses two truly positive patients.

If the threshold is lowered to $t = 0.40$, then P_1 through P_6 are predicted positive. This gives

$$TP = 4, \quad FP = 2, \quad FN = 0, \quad TN = 2.$$

Hence

$$\text{Precision} = \frac{4}{6} \approx 0.67, \qquad \text{Recall} = 1.$$

This threshold detects all positive patients, but it also sends two negative patients to additional testing.

The reason for using scores is now clear: the same model can support different decisions. If the hospital wants to avoid missing any serious case, a lower threshold is preferable. If the follow-up test is very expensive or invasive, a higher threshold may be preferable.

This is an example of precision-recall analysis - it studies how precision and recall change as the threshold changes, which creates a **tradeoff**

Definition: Precision-recall view

A *precision–recall analysis* examines how many positives we detect and how reliable the positive predictions remain as the threshold changes.

Remark: Precision-recall tradeoff

Usually for parameterized or score-based classifiers, changing the threshold controls a precision-recall tradeoff.

- If the threshold is *lowered*, more observations are predicted positive. Recall usually increases because fewer actual positives are missed, but precision may decrease because more false positives are included among the predicted positives.
- If the threshold is *raised*, the classifier becomes more conservative. Precision may improve, but recall may decrease because more true positives are missed.

1.8 ROC Curves

Definition: Receiver Operating Characteristic (ROC) curve

A *ROC curve* is a plot of true positive rate versus false positive rate across different threshold values.

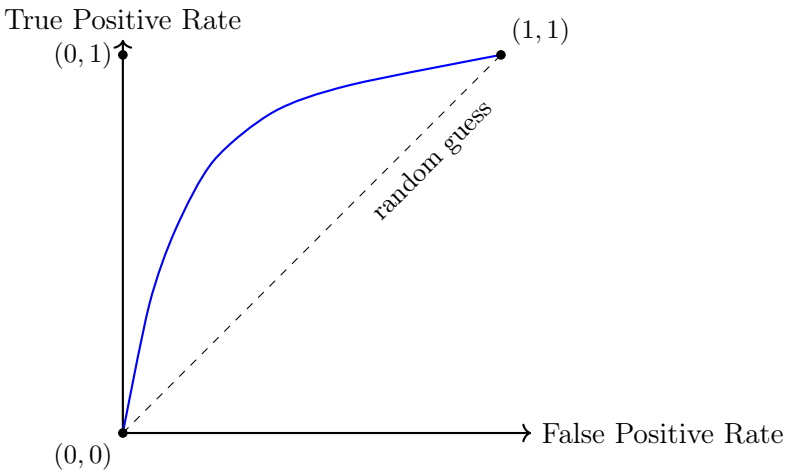


Figure 1: Schematic ROC curve. The ideal classifier is near the upper-left corner. The diagonal corresponds to random guessing.

Remark: How to read ROC space?

In ROC space:

- $(0, 0)$: TPR = 0 means no actual positives are predicted positive; FPR = 0 means no actual negatives are predicted positive. This means everything is predicted negative;
- $(1, 1)$: TPR = 1: all actual positives are predicted positive; FPR = 1: all actual negatives are also predicted positive. So the classifier labels everything as positive (everything is predicted positive);
- The upper-left corner is ideal: high TPR and low FPR;
- On the diagonal, the classifier is getting true positives at the same rate as false positives. For example, if TPR=FPR=0.3 then 30% of the real positives will be predicted positive, so and 30% of the real negatives will also be predicted positive. The classifier is not separating positives from negatives any better than chance.
- Above the diagonal: the classifier is better than random. Below the diagonal: worse than random. If we flip the predictions, it becomes better than random. Flipping means reverse the meaning of the classifier's output - it is done when there is clear

interpretation mistake (ex. positive class defined oppositely.)

Insight: Why ROC is useful?

A single confusion matrix corresponds to one threshold only. A ROC curve shows how the classifier behaves *across thresholds*. This makes ROC analysis especially useful when the operating threshold is still a design choice.

Example: Overlapping score distributions

If the positive and negative classes overlap strongly in classification score space, there is usually no threshold that perfectly separates them. Moving the threshold to catch more positives will typically also include more negatives, so both TPR and FPR change together. This is exactly the type of situation that ROC analysis is meant to summarize.

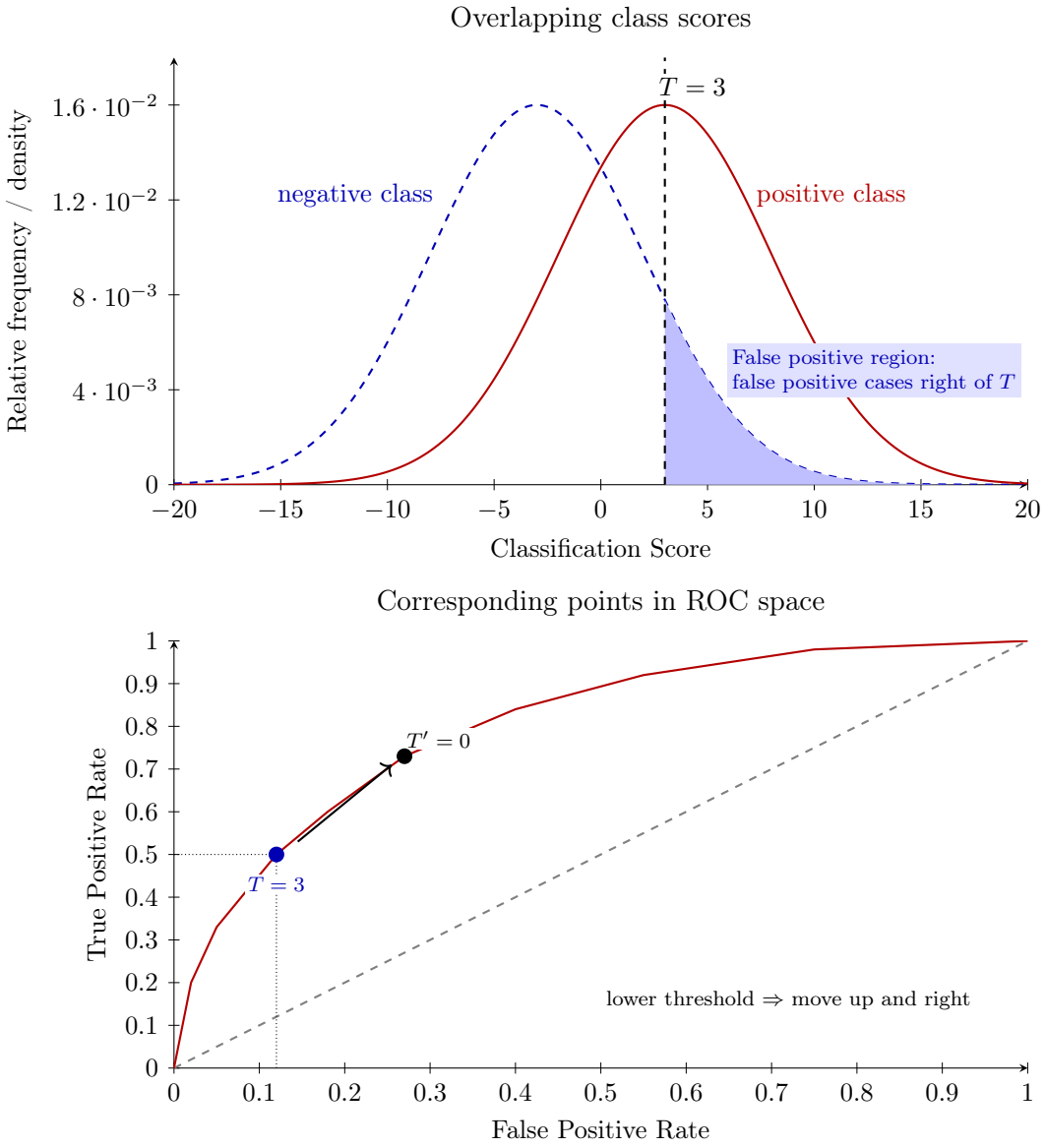


Figure 2: The shaded blue region is the false-positive region for threshold $T = 3$: these are negative cases whose score is greater than the threshold and are therefore predicted as positive.

Interpretation of the threshold values. In this example, the threshold $T = 3$ corresponds to the point $(\text{FPR}, \text{TPR}) = (0.12, 0.50)$, so

$$\text{TPR} = 0.50, \qquad \text{FNR} = 0.50, \qquad \text{FPR} = 0.12, \qquad \text{TNR} = 0.88.$$

This means that about half of the positives lie to the right of T, while only about 12% of the negatives lie to the right of T. When the threshold is lowered to $T' = 0$, more observations are classified as positive. As a result, the number of true positives increases, but the number of false positives also increases. The ROC point therefore moves upward and to the right, here from approximately $(0.12, 0.50)$ to $(0.27, 0.73)$.

1.9 How To Construct a ROC Curve?

To construct a ROC curve explicitly, one usually needs a classifier that produces a continuous-valued score for each test instance.

Example: A ranked-score example

Suppose a classifier outputs the following scores:

Instance	Score	True Class
1	0.95	+
2	0.93	+
3	0.87	−
4	0.85	−
5	0.85	−
6	0.85	+
7	0.76	−
8	0.53	+
9	0.43	−
10	0.25	+

These are already sorted from most likely positive to least likely positive. Lowering the threshold step by step produces the ROC table below.

Threshold $\geq$	TP	FP	TN	FN	TPR	FPR
1.00	0	0	5	5	0.0	0.0
0.95	1	0	5	4	0.2	0.0
0.93	2	0	5	3	0.4	0.0
0.87	2	1	4	3	0.4	0.2
0.85	3	3	2	2	0.6	0.6
0.76	3	4	1	2	0.6	0.8
0.53	4	4	1	1	0.8	0.8
0.43	4	5	0	1	0.8	1.0
0.25	5	5	0	0	1.0	1.0

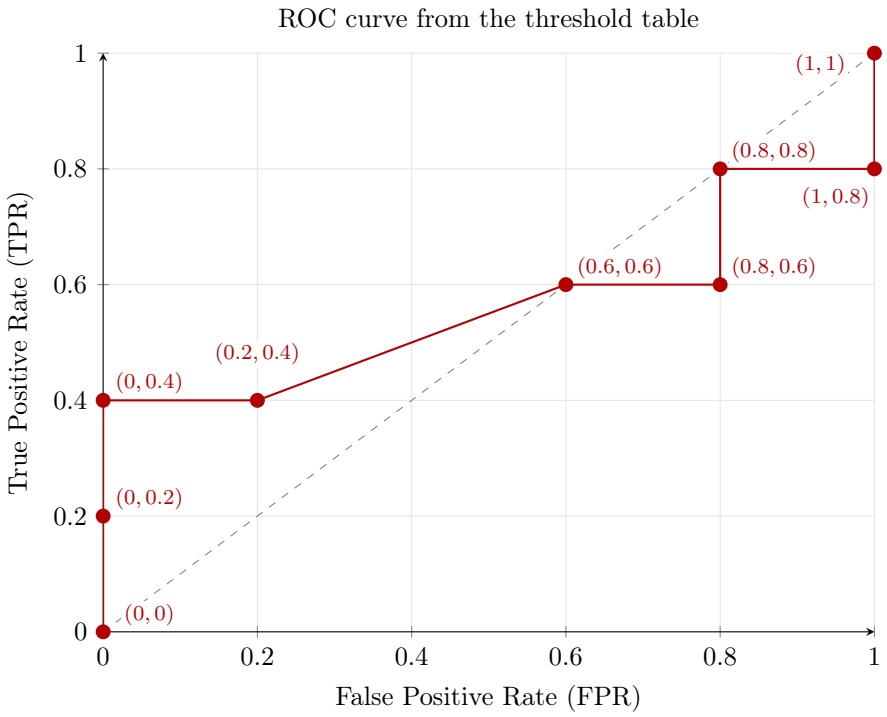


Figure 3: ROC curve obtained from the threshold table. Each red point corresponds to one threshold value.

1.10 Using ROC and AUC for Model Comparison

Sometimes two models do not have one ROC curve that lies completely above the other. One model may be better for small false positive rates, while another may be better for larger false positive rates.

Definition: AUC

The *Area Under the ROC Curve* (AUC) is the area below the ROC curve. It is a scalar summary of ROC performance across thresholds. Larger values indicate better ranking ability. An ideal classifier has AUC $= 1$, while random guessing has AUC ≈ 0.5.

Area under the ROC curve (AUC)

When the ROC curve is represented by consecutive points, this area is computed as the sum of the trapezoidal areas between neighboring points. If the ROC points are

$$(\mathrm{FPR}_1, \mathrm{TPR}_1), (\mathrm{FPR}_2, \mathrm{TPR}_2), \ldots, (\mathrm{FPR}_k, \mathrm{TPR}_k),$$

then the AUC is approximated by

$$\mathrm{AUC} \approx \sum_{i=1}^{k-1} \frac{\mathrm{TPR}_i + \mathrm{TPR}_{i+1}}{2} (\mathrm{FPR}_{i+1} - \mathrm{FPR}_i).$$

For the ROC curve in Figure 3, it can be calculated that AUC =0.48.

Remark: A practical note on ROC computation

In practice, ROC curves and AUC are usually computed from all distinct score thresholds in the test set, not from only a few manually chosen thresholds, so that the estimated curve and area reflect the model's ranking behavior more accurately.

What exactly does AUC measure? AUC measures the overall ranking quality of the classifier across all possible thresholds. A larger AUC means that the classifier tends to place positive cases above negative cases more successfully.

A **very important interpretation** is: AUC is the probability that a randomly chosen positive gets a higher score than a randomly chosen negative.

Thus y

Interpretation of typical values.

- AUC $= 1$: perfect ranking; all positives are scored above all negatives.
- AUC ≈ 0.5: essentially random ranking.
- AUC < 0.5: the ranking is systematically poor; reversing the score direction may improve performance.

How do we use AUC? AUC is useful when we want to compare models independently of one fixed threshold. If one model has a larger AUC than another, then in general it has better overall ranking ability. However, AUC does not by itself determine the best operating threshold, and it does not include the business cost of false positives and false negatives. Therefore, when comparing models in practice, AUC is often used together with threshold-specific measures and cost-sensitive considerations.

Example: Comparing two models with ROC and AUC

Suppose model M_1 and model M_2 have ROC curves that cross, as in the figure below.

- For very small false positive rates, M_1 achieves a larger true positive rate, so M_1 is preferable if false alarms must be kept very low.
- For moderate or larger false positive rates, M_2 achieves a larger true positive rate, so M_2 may be preferable in that operating region.

If we want a single threshold-independent summary, we can compare their AUC values. But if the application only allows a very small false positive rate, the model with the larger overall AUC is not automatically the practically better choice.

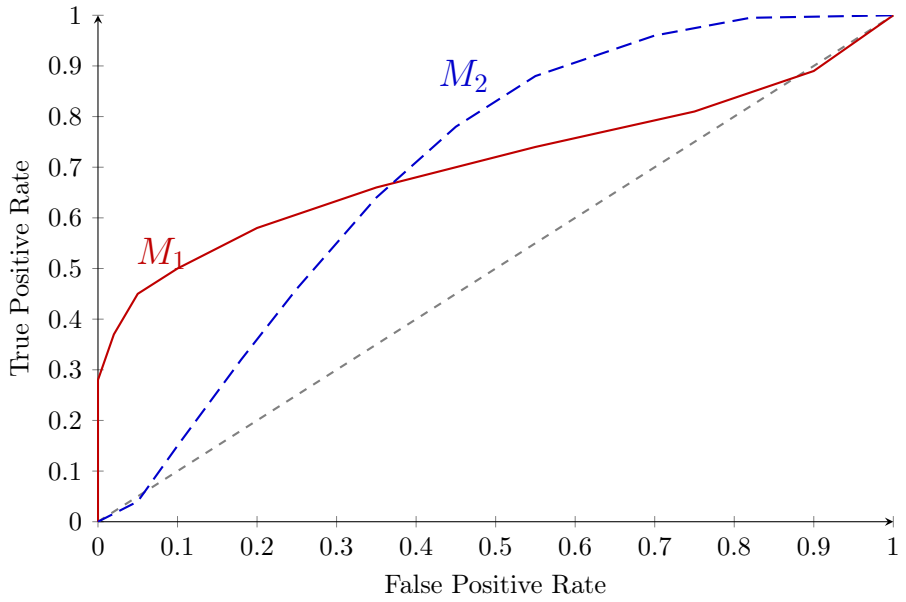


Figure 4: ROC curves for two models M_1 and M_2.

Warning: Do not use AUC mechanically

AUC is useful because it summarizes ranking performance across thresholds, but it does not by itself choose the operating threshold, encode the misclassification cost, or tell us which part of the ROC curve is most relevant for the application.

1.11 ROC Curves versus Precision–Recall Curves

Both ROC and precision–recall views study threshold behavior, but they emphasize different questions.

Remark: ROC vs PR

- ROC focuses on the tradeoff between detection rate and false alarm rate.
- Precision-recall focuses on the tradeoff between detecting positives and keeping positive predictions reliable.

In strongly imbalanced problems where the positive class is rare and important, the precision–recall view is often especially informative.

1.12 Handling the Imbalanced-Class Problem

There are three broad responses to imbalance:

1. **class-based ordering**;
2. **cost-sensitive classification**;
3. **sampling-based approaches**.

Remark: Class-based ordering

Some algorithms can explicitly prioritize the rare class. For example, rule-based systems may give higher priority to rules that detect the rare class. The main idea is simple: do not let the majority class dominate every decision merely because it is frequent.

1.13 Cost-Sensitive Classification

In many applications, false positives and false negatives do not have the same cost. For example:

- missing a cancer case may be much worse than sending an extra patient to screening;
- failing to detect fraud may be much more costly than incorrectly flagging a normal transaction;
- blocking a legitimate user may be inconvenient, but failing to block an attack may be much worse.

Definition: Cost-sensitive classification

Cost-sensitive classification is classification in which different prediction outcomes have different costs, and model decisions should take those costs into account.

Cost matrix

A cost matrix assigns a cost to each actual/predicted combination.

	Predicted Yes	**Predicted No**
Actual Yes	$C(\text{Yes}, \text{Yes})$	$C(\text{Yes}, \text{No})$
Actual No	$C(\text{No}, \text{Yes})$	$C(\text{No}, \text{No})$

If $f(i, j)$ denotes the number of class-i observations predicted as class j, then the total classification cost is

$$\text{Total cost} = \sum_i \sum_j C(i, j)\, f(i, j).$$

Example: Higher accuracy may still mean worse cost

Suppose we classify 1000 transactions and missing a truly positive case costs 20 units, while falsely flagging a negative case costs 1 unit.
Assume model M_1 has

$$TP = 40, \quad FN = 10, \quad FP = 190, \quad TN = 760.$$

Its accuracy is

$$\frac{40 + 760}{1000} = 0.80,$$

and its total cost is

$$10 \cdot 20 + 190 \cdot 1 = 390.$$

Now assume model M_2 has

$$TP = 10, \quad FN = 40, \quad FP = 60, \quad TN = 890.$$

Its accuracy is

$$\frac{10 + 890}{1000} = 0.90,$$

which is higher, but its total cost is

$$40 \cdot 20 + 60 \cdot 1 = 860.$$

So M_2 is more accurate, yet much worse from the practical point of view because it misses too many expensive positive cases.

Remark: When is accuracy aligned with cost?

Accuracy is proportional to a cost function only under a special symmetric situation: the two types of misclassification have equal cost and the two correct decisions have equal cost. Once these assumptions fail, maximizing accuracy is no longer the same as minimizing cost.

A more formal decision rule

Let us have the following notation:

- i - indexes the *true class* that the observation might actually belong to;
- j - indexes a *candidate class* that we could predict;
- $C(i, j)$ - is the cost of predicting class j when the true class is i;
- $p(i \mid x)$ is the posterior probability that the true class is i, given the observed record x.

So for each possible prediction j, we compute its expected cost:

$$\sum_i p(i \mid x)\, C(i, j).$$

Then we compare these expected costs across all possible predicted classes j. The chosen class is the one with the smallest expected cost. That chosen class is denoted by k:

$$k = \arg\min_j \sum_i p(i \mid x)\, C(i, j).$$

Thus k is prediction class that minimizes the expected classification cost.

Example: Two-class cost-sensitive rule

In the binary case, let the two classes be denoted by 0 and 1. For a test record x, we compute the expected cost of predicting each class.
If we predict class 1, the expected cost is

$$\text{Cost}(1) = p(1 \mid x)\, C(1, 1) + p(0 \mid x)\, C(0, 1).$$

If we predict class 0, the expected cost is

$$\text{Cost}(0) = p(1 \mid x)\, C(1, 0) + p(0 \mid x)\, C(0, 0).$$

We then compare these two quantities and choose the class with the smaller expected cost. Thus, we predict class 1 if

$$\text{Cost}(1) < \text{Cost}(0),$$

and otherwise we predict class 0.
If we further assume that correct classifications have zero cost, that is,

$$C(1, 1) = C(0, 0) = 0,$$

then the rule simplifies, but the decision threshold is still not automatically equal to 0.5; instead, it depends on the relative costs of the two types of mistakes.
This shows that in cost-sensitive classification, the final prediction is not determined only by which class has the larger posterior probability, but also by how serious the consequences of each possible error are.

Insight: Why this matters?

The usual posterior-probability threshold rule is only one special case. Once costs differ, the threshold should usually change as well.

1.14 Practical examples: when the best statistical model is not the best business choice

In real applications, model evaluation is not based only on classical predictive measures. A model can look stronger according to accuracy, recall, F_1, or even AUC, but still not be the model that the company deploys in production. The reason is that every prediction can trigger a business action, and those actions have costs, risks, operational consequences, and long-term effects on customers.

Example> Credit card fraud detection

A well-known public dataset for this problem is the Credit Card Fraud Detection dataset (ULB / Kaggle). It contains highly imbalanced transaction data, where fraudulent transactions are very rare compared with legitimate ones.

Suppose a company compares two fraud-detection models:

- **Model A** has higher recall and better AUC.
- **Model B** has slightly lower recall, but far fewer false positives.

At first sight, Model A may seem better because it detects more fraud. However, each false positive in this setting may lead to: a legitimate transaction being blocked, manual review by staff, customer dissatisfaction, abandoned purchases; future loss of trust and revenue.

So even if Model A performs better on several classical measures, the company may still choose Model B if Model A creates too many false alarms. In practice, the company is not only evaluating whether the model predicts well; it is evaluating whether the *decisions triggered by the model* are financially acceptable (business loss is calculated).

This example is important because it shows that a model with stronger statistical performance may still be rejected if it harms customer experience or reduces profit.

1.15 Sampling-Based Approaches

Definition: Sampling-based approaches

Sampling-based approaches modify the class distribution in the training set, for example by:

- *undersampling* the majority class;
- *oversampling* the minority class.

Undersampling

Undersampling removes some majority-class observations from the training set.

Possible advantage: training becomes more balanced and sometimes simpler.

Possible disadvantage: useful majority-class information may be discarded.

Oversampling

Oversampling increases the representation of the minority class in the training set.

Possible advantage: the model sees more minority-class examples and may detect the rare class better.

Possible disadvantage: naive oversampling may encourage overfitting or simply duplicate information.

Remark: A practical note on undersampling and oversampling

Undersampling is often useful when the majority class is extremely large and somewhat redundant, because it can make training faster and force the classifier to pay more attention to the rare class. Oversampling is often useful when the minority class is too small to be learned well, because it increases its presence during training.
In both cases, the resampling step should be applied only to the *training data* (or separately inside each cross-validation fold), not to the validation or test data. Otherwise, the evaluation may become misleading.

Warning: Sampling is not a complete solution

Sampling changes the training distribution, but it does not automatically fix poor evaluation design, leakage, or inappropriate choice of metric. Sampling must be combined with proper validation and problem-appropriate evaluation.

1.16 Which Measure Should We Use?

There is no universal best measure. The choice should follow the practical goal:

- if the classes are roughly balanced and error types matter similarly, accuracy may be acceptable;
- if missing positives is dangerous, emphasize recall;
- if false alarms are expensive or disruptive, emphasize precision or specificity;
- if both missing positives and false alarms matter, F_1 is often useful;
- if the threshold itself is a design choice, use ROC or precision–recall analysis;
- if decision consequences are unequal, use cost-sensitive evaluation rather than plain accuracy.

Warning: Do not compare classifiers vaguely

A statement such as "model A is better than model B" is incomplete unless one also states *with respect to which measure* and *for which practical goal.*

Example: Same data, different question, different measure

Suppose we have the same hospital dataset of patients arriving for possible diagnosis.

- If the question is "Can we miss as few true cases as possible in first-stage screening?", recall is especially important.
- If the question is "Can we reduce unnecessary specialist referrals?", precision and specificity become more important.
- If the question is "Can we rank patients so that the highest-risk patients are checked first?", ROC/AUC or precision–recall analysis becomes especially relevant.

The dataset is the same, but the preferred evaluation measure changes because the practical question changes.

1.17 What To Do in Practice

Remark: Where do these metrics enter the workflow?

In practice, these measures are used at different stages:

- on the **training data**, some algorithms optimize a surrogate objective related to classification quality, while we still monitor whether the model is learning the intended signal;
- on a **validation set** (or in cross-validation), we compare models, tune hyperparameters, and often choose thresholds using measures such as recall, precision, F_1, AUC, or cost;
- on the final **test set**, we report the chosen measures once, as an honest estimate of future performance.

So these metrics are not only descriptive numbers at the end. They guide training decisions, validation choices, and final evaluation.

Do's

- Report the confusion matrix, not only one summary number.
- For imbalanced data, do not report accuracy alone.
- Choose the evaluation metric to match the practical goal.
- When models produce scores, examine threshold behavior with ROC or precision–recall analysis.
- If costs differ, use cost-sensitive evaluation or a cost-aware decision rule.
- Use stratified validation or cross-validation when class imbalance is substantial.
- In rare-event problems, think explicitly about which errors are practically tolerable and which are not.

Don'ts

- Do not say "this classifier is best" without saying according to which measure.
- Do not choose a threshold on the test set and then report that same performance as the final unbiased result.
- Do not assume that the highest-accuracy model is automatically the best one for the application.
- Do not ignore the consequences of false negatives and false positives.
- Do not assume that sampling alone fixes an imbalanced-class problem.

Insight: Three levels of thinking

After this lecture, classifier quality should be thought about on three levels:

1. **Label accuracy:** how often is the predicted class correct?
2. **Ranking/threshold quality:** how well are positives separated from negatives as the threshold changes?
3. **Decision quality:** what is the practical consequence or cost of the resulting errors?

A strong classifier should be judged at the level that matches the real goal of the application.

1.18 Summary

A classifier is not automatically good just because it has high accuracy. On imbalanced data, a model can achieve excellent accuracy and still fail completely on the class that matters most.

Precision, recall, specificity, false positive rate, and F_1 each capture different aspects of performance. No single measure is best for every task. When classifiers produce scores rather than only hard labels, threshold-based evaluation becomes important, especially through ROC and precision–recall analysis.

Finally, classification quality depends not only on predictive correctness but also on practical consequences. In many applications, false positives and false negatives have different costs, and therefore cost-sensitive thinking is necessary.

The central lesson is:

> A good classifier is not the one that looks best under one generic measure. It is the one that performs well according to the actual goal, uncertainty, and consequences of the task.

1.19 Exercises

1. Explain why a classifier that predicts every observation as the majority class can still have very high accuracy.

2. A test set produces the following confusion matrix:

	Predicted Yes	**Predicted No**
Actual Yes	18	2
Actual No	30	450

   Compute accuracy, precision, recall, specificity, false positive rate, and F_1 score.

3. Two classifiers have the same accuracy, but one has much larger recall and much smaller precision. Give a practical situation in which that classifier may still be preferred.

4. Why can two classifiers with the same precision occupy different locations in ROC space?

5. What does the point $(0, 0)$ mean in ROC space? What about $(1, 1)$? Why is the diagonal associated with random guessing?

6. Why is AUC useful, and why is it still not enough by itself for choosing the final operating point of a classifier?

7. Suppose false negatives are much more expensive than false positives. Would you usually want a higher threshold or a lower threshold for predicting the positive class? Explain the expected effect on recall and false positives.

8. Explain why maximizing accuracy and minimizing classification cost are the same thing only under special assumptions on the cost matrix.

9. Give one possible advantage and one possible disadvantage of undersampling/oversampling.

10. Cost-sensitive credit approval with the German Credit dataset.
    Use the Statlog (German Credit Data) dataset from the UCI Machine Learning Repository. The dataset describes loan applicants by a set of financial and personal attributes and

classifies them as good or bad credit risks. The dataset comes with the following cost matrix:

	Predict good	Predict bad
True good	0	1
True bad	5	0

Train at least two classification models on this dataset. For each model:

- report the confusion matrix;
- compute accuracy, precision, recall, and total classification cost under the given cost matrix;
- compare the models from both a standard predictive perspective and a cost-sensitive perspective.

Based on your results, decide which model should be preferred by the bank, and explain why this decision may differ from the choice based only on accuracy.